SMART STUDENT DEADLINE MANAGER

Project Documentation

Project Type: Individual Go Project	Current Stage: Core functionality in progress

1. Project Overview

Smart Student Deadline Manager is a task and deadline management application being developed to help students organise their academic responsibilities in one place. Students regularly handle assignments, practical work, projects, tests and examinations, each with different due dates and levels of importance. When these tasks are tracked manually, it can become difficult to remember deadlines and decide what should be completed first.

The project aims to provide a simple system in which a student can record academic tasks, view their details and keep track of their pending work. The application is being built using the Go programming language. At the current development stage, the basic task model and the operations for adding and viewing tasks have been implemented. Additional management and deadline-related features are planned for the next stages.

2. Objectives

- Provide a central place to record assignments, projects, exams and other academic tasks.
- Make it easier to view task details, including the subject, due date and priority.
- Help students become more organised and reduce the chances of missing important deadlines.
- Build towards identifying upcoming and overdue tasks so students can plan their workload.
- Apply Go programming concepts such as structs, functions, slices, packages and file handling.

3. Key Features

Features implemented so far

- **Add a task:** The user can enter task information such as an ID, title, description, subject, task type, due date and priority. A newly added task is assigned the status “Pending”.
- **View tasks:** The application displays the tasks that have been added, along with their available details. It also displays the total number of tasks currently stored during the running session.
- **Structured task data:** Task information is represented through a Go struct, which keeps the fields for each task organised in a consistent format.

Planned / currently under development

- **Update and delete tasks:** Allow users to edit task information or remove tasks that are no longer required.
- **Deadline tracking:** Identify tasks that are approaching their due dates and tasks that have become overdue.
- **Priority support:** Improve task prioritisation so students can decide which work needs attention first.
- **Data persistence:** Save task records in a JSON file so that information can be loaded again after the application is closed. At present, tasks are held in memory and are not retained after restart.
- **Workload overview and reminders:** Provide a clearer view of pending work and develop deadline-monitoring functionality.
- **User interface:** A user-friendly interface is also being worked on. It is not completed or integrated yet; the current version is being developed through a basic command-line menu.

4. Technologies and Tools Used

- **Go (Golang):** Main programming language used to develop the application and its task-management logic.
- **Visual Studio Code:** Code editor used to create, organise and work on the project files.

- **Go modules (go.mod):** Used to define the Go module and manage the project structure.
- **JSON (planned for storage):** Selected as the intended file format for saving task data; persistent JSON storage is not yet complete.

5. Implementation and Working

The project is organised into separate Go files and packages so that different responsibilities can be maintained independently. The main program provides the entry point and menu flow, while the model defines the structure of a task. Service files contain the task-related operations, and the storage package is intended to handle saving and retrieving task data.

File / Package	Responsibility
main.go	Starts the application and presents the current menu-driven interaction.
models/task.go	Defines the Task data structure and its fields.
services/task.go	Contains task operations, including adding and viewing tasks.
services/monitor.go	Reserved for deadline-monitoring functionality being developed.
storage/storage.go	Provides the structure for storage-related functionality; persistent saving/loading is still to be completed.
go.mod	Defines the Go module for the project.

When the program runs, the user interacts with the menu and selects an available operation. For adding a task, the program collects the required details and creates a Task value with a Pending status. The task is then added to an in-memory slice. When the view option is selected, the program reads the tasks from that slice and prints their information and the total task count. This means the current version demonstrates the basic create-and-view workflow, but the data is temporary and is lost when the program stops.

6. Current Status and Future Improvements

The foundational structure of the application and its basic add-task and view-task functions have been developed. Other menu operations and the complete deadline-management workflow are still in progress. The user interface is also being designed and developed, and will be integrated after the current core functionality is expanded.

The next development steps are to complete update and delete operations, implement JSON-based data storage, add upcoming/overdue deadline checks and improve priority handling. After these functions are stable, the interface and testing can be completed. The overall goal is to turn the current basic Go application into a practical tool that helps students manage academic work more systematically.

Development note: *This document describes the present implementation honestly and separates completed work from planned features.*